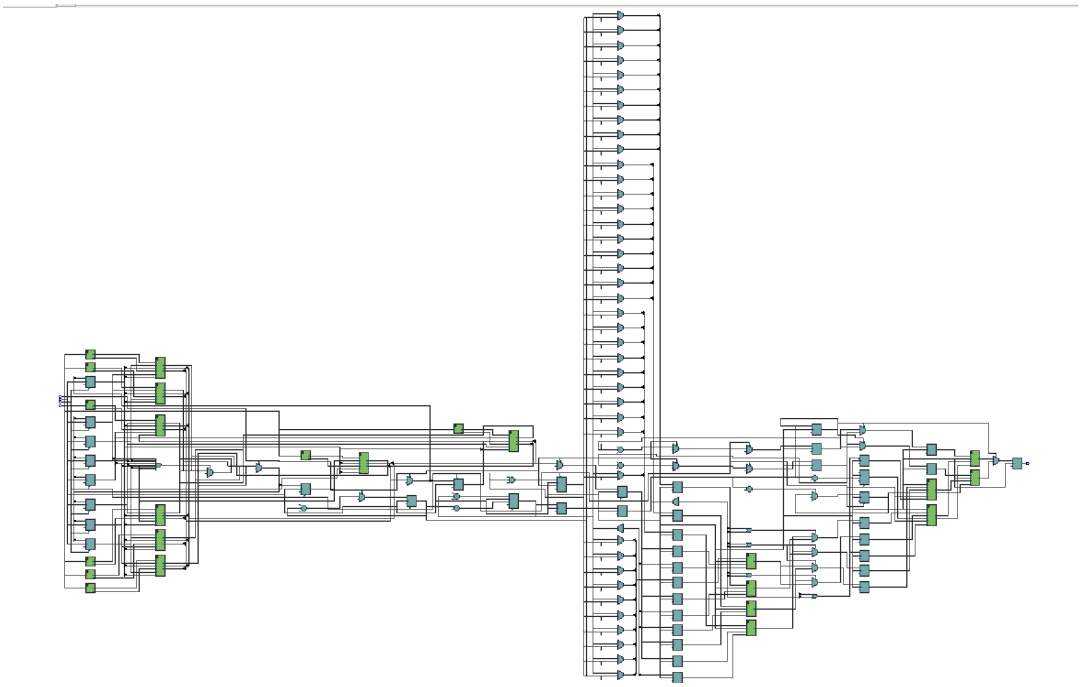
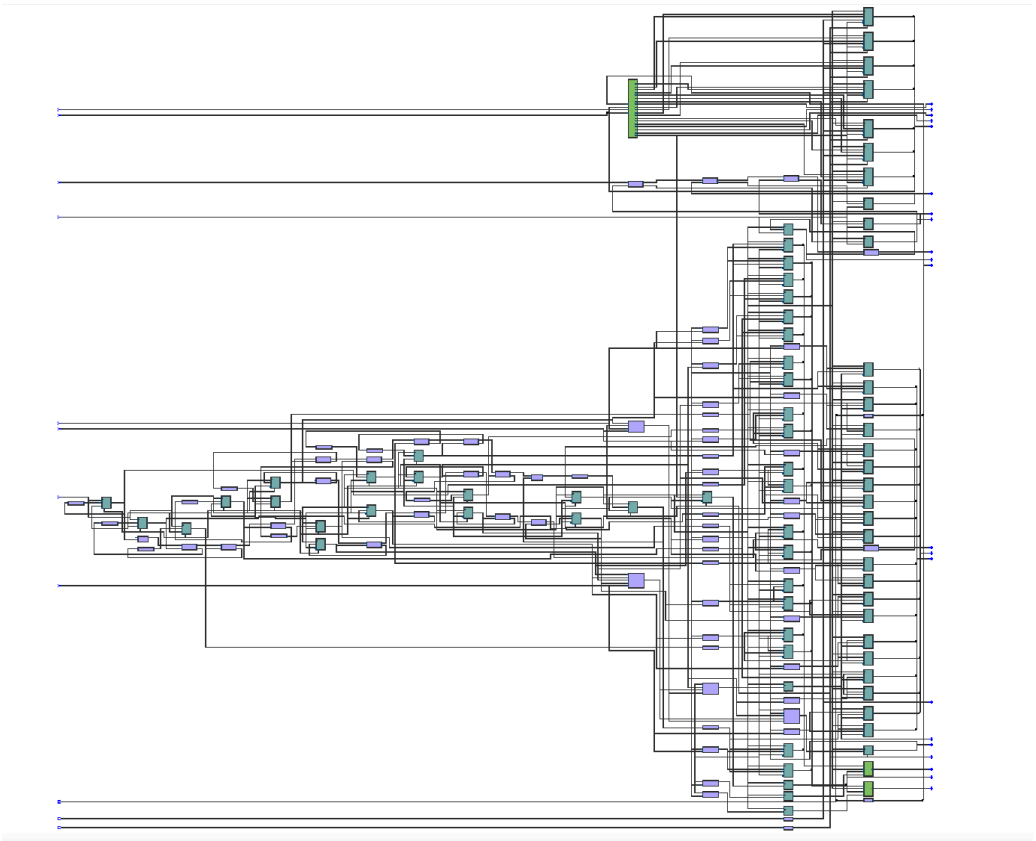
Qinyi Li(U10464059)
Elisha Sengupta(A17466620)

Term Project Report Part 1

1. Synthesis(Viterbi decoder)
   a. RTL schematic



   b. Post mapping schematic



c. Resource usage

	Resource	Usage
1	Estimated ALUTs Used	355
1	-- Combinational ALUTs	355
2	-- Memory ALUTs	0
3	-- LUT_REGs	0
2	Dedicated logic registers	239
3		
4	Estimated ALUTs Unavailable	57
5		
6	Total combinational functions	355
7	Combinational ALUT usage by number of inputs	
1	-- 7 input functions	8
2	-- 6 input functions	62
3	-- 5 input functions	14
4	-- 4 input functions	10
5	-- <=3 input functions	261
8		
9	Combinational ALUTs by mode	
1	-- normal mode	179
2	-- extended LUT mode	8
3	-- arithmetic mode	168
4	-- shared arithmetic mode	0
10		
11	Estimated ALUT/register pairs used	490
12		
13	Total registers	239
14		
15		
16	I/O pins	6
17	Total MLAB memory bits	0

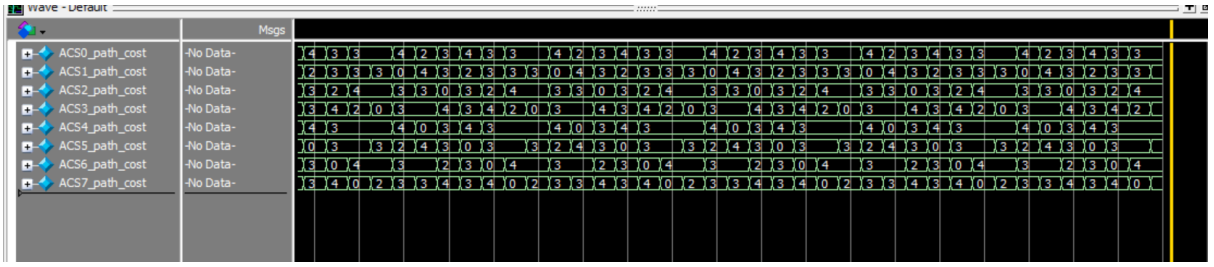
2. Simulation

a. transcript

```
# yaa! in = 1, out = 1, w_ct =         249, err = 00
# yaa! in = 1, out = 1, w_ct =         250, err = 00
# yaa! in = 1, out = 1, w_ct =         251, err = 00
# yaa! in = 1, out = 1, w_ct =         252, err = 00
# yaa! in = 1, out = 1, w_ct =         253, err = 00
# yaa! in = 1, out = 1, w_ct =         254, err = 00
# yaa! in = 1, out = 1, w_ct =         255, err = 00
# corrupted_bits =           0, OUT: good =         256, bad =           0
# ** Note: $stop    : C:/Users/12814/Downloads/thenewestcodewhichpasstheautograder/viterbi_tx_rx_tb.sv(91)
#    Time: 1045 us  Iteration: 0  Instance: /viterbi_tx_rx_tb
# Break in Module viterbi_tx_rx_tb at C:/Users/12814/Downloads/thenewestcodewhichpasstheautograder/viterbi_tx_rx_tb.sv line 91
```

It's shown in the transcript that the decoder can successfully decode the input values when there is no error injection.

b. simulation waveform



There is always one path whose path_cost is 0 at each cycle, indicating that the Add-Compare-Select (ACS) units are correctly tracking the ideal (zero-error) path through the trellis.

3.Explanations

a. **explain how this encoder works**

This encoder implements a convolutional encoder using a FSM with 3-bit state memory, resulting in 8 possible states.

For each input bit (d_in), the FSM transitions to a new state and produces a 2-bit output (d_out_reg), based on the current state and input.

On each rising clock edge (if enable_i is asserted and rst is inactive), it updates the state by right shifting the previous bits (cstate[2:1]) and inserting a new bit computed as d_in ^ cstate[1] ^ cstate[0].

The 2-bit output d_out is generated according to convolutional encoding rules: the first output bit is simply the input bit (d_out[0] = d_in), while the second is the XOR of the input and selected state bits (d_out[1] = d_in ^ cstate[2] ^ cstate[1]).

b. **explain how the decoder works**

The Viterbi decoder is a digital signal processing block designed to find the most likely sequence of states in a finite-state process, making it ideal for decoding convolutionally encoded data. The architecture is built from four primary components: Branch Metric Computation (BMC) units, Add-Compare-Select (ACS) units, Trellis Memory banks, and Trace-Back Units (TBU).

The BMC units calculate the branch metric for each possible state transition in the trellis. This metric is the Hamming distance between the 2-bit symbol received from the channel (d_in) and the ideal symbol that the encoder would have produced for that specific transition. A smaller metric indicates a closer match, meaning the transition was more likely to have occurred.

The ACS units form the core recursive engine of the decoder. For each state in the trellis, an ACS unit performs three operations:

①Add: It adds the incoming branch metric from a BMC unit to the accumulated path metric of the preceding state to calculate a new total path cost.

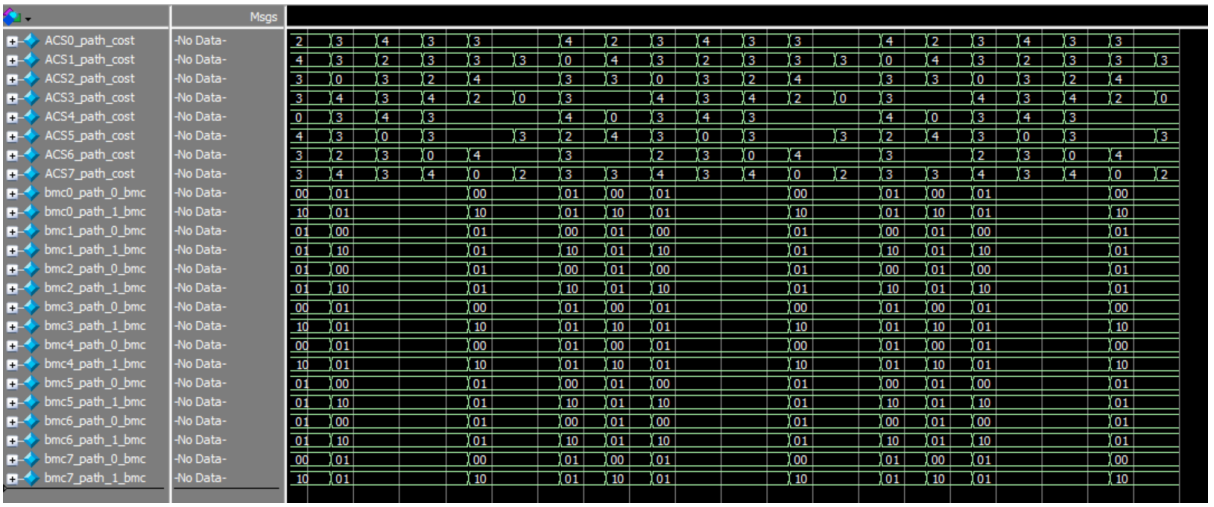
②Compare: It compares the total path costs of the two paths that converge at the current state.

③Select: It chooses the path with the minimum cost as the survivor path. The decision (selection) bit indicating which path was chosen is stored for later use.

The decoder uses four banks of memory to store the selection bits generated by the ACS units at each time step. This creates a record of survivor paths through the trellis over a specific depth. The memory is managed in a circular buffer fashion, allowing new decisions to be written continuously while older decisions are read for traceback.

The TBU is responsible for the final decoding step. It reads the selection bits from the trellis memory and, starting from a state with the lowest path cost (or a fixed end-state like state 0), traces the single most likely path backward through the trellis. Based on the decisions encountered at each step of the traceback, the TBU reconstructs the original sequence of bits that were fed into the encoder, producing the final d_out.

c. **show the minimum branch metric=0 when no errors are present**



When the decoder input exactly matches the expected output symbol for a given path, the corresponding branch metric will be 00.

d. explain how the decoder determines what sequence of bits was sent to the encoder

First, at each time step, the ACS units find the lowest-cost "survivor path" to every state and output a corresponding selection bit, which represents the most likely encoder input (0 or 1) for that path. These decision bits are stored in the trellis memory, creating a historical record of all survivor paths. Subsequently, the Trace-Back Unit (TBU) starts from a known optimal state and reads these stored selection bits backward through the memory. The sequence of selection bits encountered by the TBU as it follows the single most likely path through the trellis is the reconstructed bitstream, which, after being ordered correctly, becomes the final decoded output.

4. Codes

a. decoder

```
module decoder
(
  input          clk,
  input          rst,
  input          enable,
  input [1:0]      d_in,
  output logic     d_out);

//bmc module signals  (8 of these)
 // wire  [1:0]      bmcK_path_N_bmc;  // N=0,1,...7     N=0,1  (16 2-wire signals in all)
  wire  [1:0]      bmc0_path_0_bmc;
        wire  [1:0]      bmc0_path_1_bmc;
        wire  [1:0]      bmc1_path_0_bmc;
        wire  [1:0]      bmc1_path_1_bmc;
        wire  [1:0]      bmc2_path_0_bmc;
        wire  [1:0]      bmc2_path_1_bmc;
        wire  [1:0]      bmc3_path_0_bmc;
        wire  [1:0]      bmc3_path_1_bmc;
        wire  [1:0]      bmc4_path_0_bmc;
```

```
        wire  [1:0]      bmc4_path_1_bmc;
        wire  [1:0]      bmc5_path_0_bmc;
        wire  [1:0]      bmc5_path_1_bmc;
        wire  [1:0]      bmc6_path_0_bmc;
        wire  [1:0]      bmc6_path_1_bmc;
        wire  [1:0]      bmc7_path_0_bmc;
        wire  [1:0]      bmc7_path_1_bmc;
//ACS modules signals
  logic   [7:0]      validity;//Store path validity for each state (updated every clock
cycle)
  logic   [7:0]      selection;// Store selection signal for each state
  logic   [7:0]      path_cost   [8];// Store path cost for each state
  wire    [7:0]      validity_nets;// Concatenated valid_o outputs from 8 ACS units
  wire    [7:0]      selection_nets;// Concatenated selection outputs from 8 ACS units

  //wire            ACSK_selection;  // K=0,1,...7                (i.e., 8 of these)
        wire            ACS0_selection;
        wire            ACS1_selection;
        wire            ACS2_selection;
        wire            ACS3_selection;
        wire            ACS4_selection;
        wire            ACS5_selection;
        wire            ACS6_selection;
        wire            ACS7_selection;


 // wire            ACSK_valid_o;          // K=0,1,...7
  wire            ACS0_valid_o;
        wire            ACS1_valid_o;
        wire            ACS2_valid_o;
        wire            ACS3_valid_o;
        wire            ACS4_valid_o;
        wire            ACS5_valid_o;
        wire            ACS6_valid_o;
        wire            ACS7_valid_o;

  //wire  [7:0]      ACSK_path_cost;  // K=0,1,...7
        wire  [7:0]      ACS0_path_cost;
        wire  [7:0]      ACS1_path_cost;
        wire  [7:0]      ACS2_path_cost;
        wire  [7:0]      ACS3_path_cost;
        wire  [7:0]      ACS4_path_cost;
        wire  [7:0]      ACS5_path_cost;
        wire  [7:0]      ACS6_path_cost;
        wire  [7:0]      ACS7_path_cost;
```

```
//Trelis memory write operation, pipeline delay
  logic   [1:0]      mem_bank;
  logic   [1:0]      mem_bank_Q;
  logic   [1:0]      mem_bank_Q2;
  logic              mem_bank_Q3;
  logic              mem_bank_Q4;
  logic              mem_bank_Q5;
  logic   [9:0]      wr_mem_counter;
  logic   [9:0]      rd_mem_counter;

// 4 memory banks -- address pointers          (there are 4 of these)
  //logic   [9:0]      addr_mem_K;  // K=A,B,C,D
        logic   [9:0]      addr_mem_A;
        logic   [9:0]      addr_mem_B;
        logic   [9:0]      addr_mem_C;
        logic   [9:0]      addr_mem_D;
// write enables
  //logic              wr_mem_K; // K=A,B,C,D
        logic              wr_mem_A;
        logic              wr_mem_B;
        logic              wr_mem_C;
        logic              wr_mem_D;
// data to memories
  //logic   [7:0]      d_in_mem_K;       // K=A,B,C,D
        logic   [7:0]      d_in_mem_A;
        logic   [7:0]      d_in_mem_B;
        logic   [7:0]      d_in_mem_C;
        logic   [7:0]      d_in_mem_D;
// data from memories
  //wire    [7:0]      d_o_mem_K;       // K=A,B,C,D
        wire    [7:0]      d_o_mem_A;
        wire    [7:0]      d_o_mem_B;
        wire    [7:0]      d_o_mem_C;
        wire    [7:0]      d_o_mem_D;

//Trace back module signals
  logic              selection_tbu_0;
  logic              selection_tbu_1;

  logic   [7:0]      d_in_0_tbu_0;
  logic   [7:0]      d_in_1_tbu_0;
  logic   [7:0]      d_in_0_tbu_1;
  logic   [7:0]      d_in_1_tbu_1;

  wire               d_o_tbu_0;
  wire               d_o_tbu_1;

  logic              enable_tbu_0;
```

```
  logic             enable_tbu_1;

//Display memory operations
  wire              wr_disp_mem_0;
  wire              wr_disp_mem_1;

  wire              d_in_disp_mem_0;
  wire              d_in_disp_mem_1;

  logic   [9:0]      wr_mem_counter_disp;
  logic   [9:0]      rd_mem_counter_disp;

  logic   [9:0]      addr_disp_mem_0;
  logic   [9:0]      addr_disp_mem_1;

//Branch metric calculation modules  (8 total)(Branch metric calculation)

  bmc0
bmc0_inst(.rx_pair(d_in),.path_0_bmc(bmc0_path_0_bmc),.path_1_bmc(bmc0_path
_1_bmc));
        bmc1
bmc1_inst(.rx_pair(d_in),.path_0_bmc(bmc1_path_0_bmc),.path_1_bmc(bmc1_path
_1_bmc));
        bmc1
bmc2_inst(.rx_pair(d_in),.path_0_bmc(bmc2_path_0_bmc),.path_1_bmc(bmc2_path
_1_bmc));
        bmc0
bmc3_inst(.rx_pair(d_in),.path_0_bmc(bmc3_path_0_bmc),.path_1_bmc(bmc3_path
_1_bmc));
        bmc0
bmc4_inst(.rx_pair(d_in),.path_0_bmc(bmc4_path_0_bmc),.path_1_bmc(bmc4_path
_1_bmc));
        bmc1
bmc5_inst(.rx_pair(d_in),.path_0_bmc(bmc5_path_0_bmc),.path_1_bmc(bmc5_path
_1_bmc));
        bmc1
bmc6_inst(.rx_pair(d_in),.path_0_bmc(bmc6_path_0_bmc),.path_1_bmc(bmc6_path
_1_bmc));
        bmc0
bmc7_inst(.rx_pair(d_in),.path_0_bmc(bmc7_path_0_bmc),.path_1_bmc(bmc7_path
_1_bmc));

//K=0,1,...7
/*
  bmc0   bmc0_inst(d_in,bmc0_path_0_bmc,bmc0_path_1_bmc);
        bmc0   bmc1_inst(d_in,bmc1_path_0_bmc,bmc1_path_1_bmc);
        bmc1   bmc2_inst(d_in,bmc2_path_0_bmc,bmc2_path_1_bmc);
        bmc1   bmc3_inst(d_in,bmc3_path_0_bmc,bmc3_path_1_bmc);
```

```
        bmc1   bmc4_inst(d_in,bmc4_path_0_bmc,bmc4_path_1_bmc);
        bmc1   bmc5_inst(d_in,bmc5_path_0_bmc,bmc5_path_1_bmc);
        bmc0   bmc6_inst(d_in,bmc6_path_0_bmc,bmc6_path_1_bmc);
        bmc0   bmc7_inst(d_in,bmc7_path_0_bmc,bmc7_path_1_bmc);
*/
//Add Compare Select Modules (8 copies -- note pattern in connections!!)
// i = 0, 1, ... 7       j = 0, 3, 4, 7, 1, 2, 5, 6       k = 1, 2, 5, 6, 0, 3, 4, 7  -- these create lattice butterfly connection pattern
  //ACS
ACSi(validity[j],validity[k],bmci_path_i_bmc,bmci_path_1_bmc,path_cost[j],path_cost[k],ACSi_selection,ACSi_valid_o,ACSi_path_cost);
  ACS ACS0 (
   .path_0_valid(validity[0]),
   .path_1_valid(validity[1]),
   .path_0_bmc(bmc0_path_0_bmc),
   .path_1_bmc(bmc0_path_1_bmc),
   .path_0_pmc(path_cost[0]),
   .path_1_pmc(path_cost[1]),
   .selection(ACS0_selection),
   .valid_o(ACS0_valid_o),
   .path_cost(ACS0_path_cost));
        ACS ACS1 (
   .path_0_valid(validity[3]),
   .path_1_valid(validity[2]),
   .path_0_bmc(bmc1_path_0_bmc),
   .path_1_bmc(bmc1_path_1_bmc),
   .path_0_pmc(path_cost[3]),
   .path_1_pmc(path_cost[2]),
   .selection(ACS1_selection),
   .valid_o(ACS1_valid_o),
   .path_cost(ACS1_path_cost));
  ACS ACS2 (
   .path_0_valid(validity[4]),
   .path_1_valid(validity[5]),
   .path_0_bmc(bmc2_path_0_bmc),
   .path_1_bmc(bmc2_path_1_bmc),
   .path_0_pmc(path_cost[4]),
   .path_1_pmc(path_cost[5]),
   .selection(ACS2_selection),
   .valid_o(ACS2_valid_o),
   .path_cost(ACS2_path_cost));
        ACS ACS3 (
   .path_0_valid(validity[7]),
   .path_1_valid(validity[6]),
   .path_0_bmc(bmc3_path_0_bmc),
   .path_1_bmc(bmc3_path_1_bmc),
   .path_0_pmc(path_cost[7]),
   .path_1_pmc(path_cost[6]),
```

```
    .selection(ACS3_selection),
    .valid_o(ACS3_valid_o),
    .path_cost(ACS3_path_cost));
          ACS ACS4 (
    .path_0_valid(validity[1]),
    .path_1_valid(validity[0]),
    .path_0_bmc(bmc4_path_0_bmc),
    .path_1_bmc(bmc4_path_1_bmc),
    .path_0_pmc(path_cost[1]),
    .path_1_pmc(path_cost[0]),
    .selection(ACS4_selection),
    .valid_o(ACS4_valid_o),
    .path_cost(ACS4_path_cost));
          ACS ACS5 (
    .path_0_valid(validity[2]),
    .path_1_valid(validity[3]),
    .path_0_bmc(bmc5_path_0_bmc),
    .path_1_bmc(bmc5_path_1_bmc),
    .path_0_pmc(path_cost[2]),
    .path_1_pmc(path_cost[3]),
    .selection(ACS5_selection),
    .valid_o(ACS5_valid_o),
    .path_cost(ACS5_path_cost));
   ACS ACS6 (
    .path_0_valid(validity[5]),
    .path_1_valid(validity[4]),
    .path_0_bmc(bmc6_path_0_bmc),
    .path_1_bmc(bmc6_path_1_bmc),
    .path_0_pmc(path_cost[5]),
    .path_1_pmc(path_cost[4]),
    .selection(ACS6_selection),
    .valid_o(ACS6_valid_o),
    .path_cost(ACS6_path_cost));
          ACS ACS7 (
    .path_0_valid(validity[6]),
    .path_1_valid(validity[7]),
    .path_0_bmc(bmc7_path_0_bmc),
    .path_1_bmc(bmc7_path_1_bmc),
    .path_0_pmc(path_cost[6]),
    .path_1_pmc(path_cost[7]),
    .selection(ACS7_selection),
    .valid_o(ACS7_valid_o),
    .path_cost(ACS7_path_cost));
   // validity_nets   = // same for ACSK_valid_os
   assign selection_nets = {ACS7_selection, ACS6_selection, ACS5_selection,
ACS4_selection, ACS3_selection, ACS2_selection, ACS1_selection,
ACS0_selection};
```

```
  assign validity_nets  = {ACS7_valid_o, ACS6_valid_o, ACS5_valid_o,
ACS4_valid_o, ACS3_valid_o, ACS2_valid_o, ACS1_valid_o, ACS0_valid_o};


  always @ (posedge clk, negedge rst) begin
    if(!rst)  begin
      validity         <= 8'b1;
      selection         <= 8'b0;
// clear all 8 path costs
      path_cost[0]      <= 8'd0;
                        path_cost[1]      <= 8'd0;
                        path_cost[2]      <= 8'd0;
                        path_cost[3]      <= 8'd0;
                        path_cost[4]      <= 8'd0;
                        path_cost[5]      <= 8'd0;
                        path_cost[6]      <= 8'd0;
                        path_cost[7]      <= 8'd0;
    end
    else if(!enable)   begin
      validity         <= 8'b1;
      selection         <= 8'b0;
// clear all 8 path costs
      path_cost[0]      <= 8'd0;
                        path_cost[1]      <= 8'd0;
                        path_cost[2]      <= 8'd0;
                        path_cost[3]      <= 8'd0;
                        path_cost[4]      <= 8'd0;
                        path_cost[5]      <= 8'd0;
                        path_cost[6]      <= 8'd0;
                        path_cost[7]      <= 8'd0;

    end
    else if ( path_cost[0][7] && path_cost[1][7] && path_cost[2][7] && path_cost[3][7]
&& path_cost[4][7] && path_cost[5][7] && path_cost[6][7] && path_cost[7][7])
    begin

      validity         <= validity_nets;
      selection         <= selection_nets;
      //path_cost[K]      <= 8'b01111111 & ACSK_path_cost; // K = 0, 1, ..., 7
                        path_cost[0]      <= 8'b01111111 & ACS0_path_cost;
                        path_cost[1]      <= 8'b01111111 & ACS1_path_cost;
                        path_cost[2]      <= 8'b01111111 & ACS2_path_cost;
                        path_cost[3]      <= 8'b01111111 & ACS3_path_cost;
                        path_cost[4]      <= 8'b01111111 & ACS4_path_cost;
                        path_cost[5]      <= 8'b01111111 & ACS5_path_cost;
                        path_cost[6]      <= 8'b01111111 & ACS6_path_cost;
                        path_cost[7]      <= 8'b01111111 & ACS7_path_cost;
```

```
    end
    else   begin
      validity         <= validity_nets;
      selection        <= selection_nets;

      //path_cost[K]     <= ACSK_path_cost;         // K = 0, 1, ..., 7
                         path_cost[0]     <= ACS0_path_cost;
                         path_cost[1]     <= ACS1_path_cost;
                         path_cost[2]     <= ACS2_path_cost;
                         path_cost[3]     <= ACS3_path_cost;
                         path_cost[4]     <= ACS4_path_cost;
                         path_cost[5]     <= ACS5_path_cost;
                         path_cost[6]     <= ACS6_path_cost;
                         path_cost[7]     <= ACS7_path_cost;
    end
  end

//trellis memory
  always @ (posedge clk, negedge rst) begin           // wr_mem_counter   Track write
counter
// if rst (active low) or not enabling (active high), force to 0; else, increment by 1
    if(!rst) wr_mem_counter <= 10'd0;
             else if(!enable) wr_mem_counter <= 10'd0;
             else wr_mem_counter <= wr_mem_counter + 1;
  end

  always @ (posedge clk, negedge rst) begin//Track read counter (trace back)
    if(!rst)
      rd_mem_counter <= 10'd1023;// set to max value
    else if(enable)
      rd_mem_counter <= rd_mem_counter - 1;// count down by 1
  end

  always @ (posedge clk, negedge rst)//When one bank is full at 1023, move to next
bank
    if(!rst)
      mem_bank <= 2'b0;
    else begin
      /*if(wr_mem_counter = -1  fill in the guts*/
                       if(wr_mem_counter == 10'd1023)
          mem_bank <= mem_bank + 2'b1;
    end

  always @ (posedge clk)    begin
    d_in_mem_A  <= selection;                  // k = A, B, C, D - all banks write the
same selection data
             d_in_mem_B  <= selection;
             d_in_mem_C  <= selection;
```

```
                d_in_mem_D  <= selection;
  end

// memory bank management: always write to one, read from two others, keep address at 0 (no writing) for fourth one
  always @ (posedge clk)       // in each case, the memory bank w/ the wr_mem_counter needs a write enable; all others = 0 e.g. 0-1023: Write to Bank A, traceback read from Bank B&D 1024-2047: Write to Bank B, traceback read from Bank A&C 2048-3071: Write to Bank C, traceback read from Bank B&D 3072-4095: Write to Bank D, traceback read from Bank A&C
  begin
                case(mem_bank)
      2'b00: begin                // write to A, clear C, read from others
        wr_mem_A <= 1'b1;
        wr_mem_B <= 1'b0;
        wr_mem_C <= 1'b0;
        wr_mem_D <= 1'b0;
        addr_mem_A <= wr_mem_counter;
        addr_mem_B <= rd_mem_counter;
        addr_mem_C <= 10'd0;
        addr_mem_D <= rd_mem_counter;
      end
      2'b01: begin                // write to B, clear D, read from others
        wr_mem_A <= 1'b0;
        wr_mem_B <= 1'b1;
        wr_mem_C <= 1'b0;
        wr_mem_D <= 1'b0;
        addr_mem_A <= rd_mem_counter;
        addr_mem_B <= wr_mem_counter;
        addr_mem_C <= rd_mem_counter;
        addr_mem_D <= 10'd0;
      end
      2'b10: begin                        // write to C, clear A, read from others
        wr_mem_A <= 1'b0;
        wr_mem_B <= 1'b0;
        wr_mem_C <= 1'b1;
        wr_mem_D <= 1'b0;
        addr_mem_A <= 10'd0;
        addr_mem_B <= rd_mem_counter;
        addr_mem_C <= wr_mem_counter;
        addr_mem_D <= rd_mem_counter;
      end
      2'b11: begin           // write to D, clear B, read from others
        wr_mem_A <= 1'b0;
        wr_mem_B <= 1'b0;
        wr_mem_C <= 1'b0;
        wr_mem_D <= 1'b1;
        addr_mem_A <= rd_mem_counter;
```

```
            addr_mem_B <= 10'd0;
            addr_mem_C <= rd_mem_counter;
            addr_mem_D <= wr_mem_counter;
          end
        endcase
    end

//Trelis memory module instantiation

    mem   trelis_mem_A              (
      .clk,
      .wr  (wr_mem_A),
      .addr(addr_mem_A),
      .d_i (d_in_mem_A),
      .d_o (d_o_mem_A)
    );
/* likewise for trelis_memB, C, D
*/
    mem   trelis_mem_B              (
      .clk(clk),
      .wr  (wr_mem_B),
      .addr(addr_mem_B),
      .d_i (d_in_mem_B),
      .d_o (d_o_mem_B)
    );

    mem   trelis_mem_C              (
      .clk(clk),
      .wr  (wr_mem_C),
      .addr(addr_mem_C),
      .d_i (d_in_mem_C),
      .d_o (d_o_mem_C)
    );

    mem   trelis_mem_D              (
      .clk(clk),
      .wr  (wr_mem_D),
      .addr(addr_mem_D),
      .d_i (d_in_mem_D),
      .d_o (d_o_mem_D)
    );


//Trace back module operation

    always @(posedge clk)
/* create mem_bank, mem_bank_Q1, mem_bank_Q2 pipeline */
    begin
```

```
  mem_bank_Q  <= mem_bank;     // Delay by 1 clock
  mem_bank_Q2 <= mem_bank_Q;   // Delay by 2 clocks
end

always @ (posedge clk, negedge rst)
  if(!rst)
       enable_tbu_0   <= 1'b0;
  else if(mem_bank_Q2==2'b10)
       enable_tbu_0   <= 1'b1;

always @ (posedge clk, negedge rst)
  if(!rst)
       enable_tbu_1   <= 1'b0;
  else if(mem_bank_Q2==2'b11)
       enable_tbu_1   <= 1'b1;

always @ (posedge clk)
  case(mem_bank_Q2)
    2'b00:       begin
      d_in_0_tbu_0   <= d_o_mem_D;
      d_in_1_tbu_0   <= d_o_mem_C;

      d_in_0_tbu_1   <= d_o_mem_C;
      d_in_1_tbu_1   <= d_o_mem_B;

      selection_tbu_0<= 1'b0;
      selection_tbu_1<= 1'b1;

    end
    2'b01:       begin
      d_in_0_tbu_0   <= d_o_mem_D;
      d_in_1_tbu_0   <= d_o_mem_C;

      d_in_0_tbu_1   <= d_o_mem_A;
      d_in_1_tbu_1   <= d_o_mem_D;

      selection_tbu_0<= 1'b1;
      selection_tbu_1<= 1'b0;
    end
    2'b10:       begin
      d_in_0_tbu_0   <= d_o_mem_B;
      d_in_1_tbu_0   <= d_o_mem_A;

      d_in_0_tbu_1   <= d_o_mem_A;
      d_in_1_tbu_1   <= d_o_mem_D;

      selection_tbu_0<= 1'b0;
      selection_tbu_1<= 1'b1;
```

```
        end
        2'b11:      begin
          d_in_0_tbu_0   <= d_o_mem_B;
          d_in_1_tbu_0   <= d_o_mem_A;

          d_in_0_tbu_1   <= d_o_mem_C;
          d_in_1_tbu_1   <= d_o_mem_B;

          selection_tbu_0<= 1'b1;
          selection_tbu_1<= 1'b0;
        end
      endcase

//Trace-Back modules instantiation

   tbu tbu_0   (
     .clk,
     .rst,
     .enable(enable_tbu_0),
     .selection(selection_tbu_0),
     .d_in_0(d_in_0_tbu_0),
     .d_in_1(d_in_1_tbu_0),
     .d_o(d_o_tbu_0),
     .wr_en(wr_disp_mem_0)
   );

/* analogous for tbu_1
*/
   tbu tbu_1   (
     .clk(clk),
     .rst(rst),
     .enable(enable_tbu_1),
     .selection(selection_tbu_1),
     .d_in_0(d_in_0_tbu_1),
     .d_in_1(d_in_1_tbu_1),
     .d_o(d_o_tbu_1),
     .wr_en(wr_disp_mem_1)
   );


//Display Memory modules Instantioation
//   d_in_disp_mem_K   =  d_o_tbu_K;  K=0,1
   assign d_in_disp_mem_0 = d_o_tbu_0;
   assign d_in_disp_mem_1 = d_o_tbu_1;


  mem_disp   disp_mem_0      (
     .clk(clk),
```

```
      .wr(wr_disp_mem_0),
      .addr(addr_disp_mem_0),
      .d_i(d_in_disp_mem_0),
      .d_o(d_o_disp_mem_0)
    );
/* analogous for disp_mem_1
*/
     mem_disp   disp_mem_1     (
      .clk(clk),
      .wr(wr_disp_mem_1),
      .addr(addr_disp_mem_1),
      .d_i(d_in_disp_mem_1),
      .d_o(d_o_disp_mem_1)
    );


// Display memory module operation
   always @ (posedge clk)
      mem_bank_Q3 <= mem_bank_Q2[0];

   always @ (posedge clk)
      if(!rst)
         wr_mem_counter_disp  <= 10'd2; //min value + 2
      else if(!enable)
         wr_mem_counter_disp  <= 10'd2;//same
      else
         wr_mem_counter_disp <= wr_mem_counter_disp - 1;//       decrement
wr_mem_counter_disp

   always @ (posedge clk)
      if(!rst)
         rd_mem_counter_disp  <= 10'd1021; //max value - 2
      else if(!enable)
         rd_mem_counter_disp  <= 10'd1021;//same
      else
                        rd_mem_counter_disp <= rd_mem_counter_disp + 1; // increment
rd_mem_counter_disp

   always @ (posedge clk)
           begin
      // if !mem_bank_Q3
                     if (!mem_bank_Q3)
         begin
            addr_disp_mem_0   <= rd_mem_counter_disp;
            addr_disp_mem_1   <= wr_mem_counter_disp;
         end
     //  else:        swap rd and wr
               else begin
```

```
        addr_disp_mem_0   <= wr_mem_counter_disp;
        addr_disp_mem_1   <= rd_mem_counter_disp;
      end
    end

    always @ (posedge clk)      begin
  /* pipeline mem_bank_Q3 to Q4 to Q5
   also  d_out = d_o_disp_mem_i
     i = mem_bank_Q5 */
            mem_bank_Q4 <= mem_bank_Q3;
      mem_bank_Q5 <= mem_bank_Q4;
      if (mem_bank_Q5)
        d_out <= d_o_disp_mem_1;
      else
        d_out <= d_o_disp_mem_0;
    end


  endmodule
```

b. bmc0

```
  /* For our constraint length = 3, there will be 2**3 = 8
  of these, but unlike ACS, not all are identical.
  */

  module bmc0                              // branch metric computation
  (
    input    [1:0] rx_pair,
    output   [1:0] path_0_bmc,
    output   [1:0] path_1_bmc);

  //Fill in the guts per BMC instructions
    logic tmp00, tmp01;
    logic tmp10, tmp11;

          assign tmp00 = rx_pair[0];
          assign tmp01 = rx_pair[1];
          assign tmp10 = !tmp00;
          assign tmp11 = !tmp01;

          assign path_0_bmc[1] = tmp00 & tmp01;
          assign path_0_bmc[0] = tmp00 ^ tmp01;
          assign path_1_bmc[1] = tmp10 & tmp11;
          assign path_1_bmc[0] = tmp10 ^ tmp11;


  endmodule
```

c. bmc1

```
  /* For our constraint length = 3, there will be 2**3 = 8
```

```
of these, but unlike ACS, not all are identical.
*/

module bmc1                              // branch metric computation
(
  input    [1:0] rx_pair,
  output   [1:0] path_0_bmc,
  output   [1:0] path_1_bmc);

//Fill in the guts per BMC instructions
  logic tmp00, tmp01;
  logic tmp10, tmp11;

        assign tmp00 = rx_pair[0];
        assign tmp01 = !rx_pair[1];
        assign tmp10 = !tmp00;
        assign tmp11 = !tmp01;

        assign path_0_bmc[1] = tmp00 & tmp01;
        assign path_0_bmc[0] = tmp00 ^ tmp01;
        assign path_1_bmc[1] = tmp10 & tmp11;
        assign path_1_bmc[0] = tmp10 ^ tmp11;


endmodule
```

d. acs

```
module ACS                               // add-compare-select
(  input       path_0_valid,
  input       path_1_valid,
  input [1:0] path_0_bmc,                // branch metric computation
  input [1:0] path_1_bmc,
  input [7:0] path_0_pmc,                                  // path metric
computation(before)
  input [7:0] path_1_pmc,

  output logic       selection,
  output logic       valid_o,
  output logic     [7:0] path_cost);

  wire  [7:0] path_cost_0;                        // branch metric + path metric
  wire  [7:0] path_cost_1;

// Fill in the guts per ACS instructions
  assign path_cost_0 = path_0_pmc + path_0_bmc;
        assign path_cost_1 = path_1_pmc + path_1_bmc;

 /*  always_comb begin
      case ({path_1_valid, path_0_valid})
```

```
        2'b00: begin
           // both path are invalid
           valid_o   = 1'b0;
           selection = 1'b0;
           path_cost = 8'd0;
        end
        2'b01: begin
           // path1 is valid
           valid_o   = 1'b1;
           selection = 1'b0;
           path_cost = path_cost_0;
        end
        2'b10: begin
           // path 0 is valid
           valid_o   = 1'b1;
           selection = 1'b1;
           path_cost = path_cost_1;
        end
        2'b11: begin
        // both are valid
        valid_o = 1'b1;
        if (path_cost_0 > path_cost_1) begin
           selection = 1'b1;
           path_cost = path_cost_1;
        end else begin
           selection = 1'b0;
           path_cost = path_cost_0;
        end
     end
        default: begin
           valid_o   = 1'b0;
           selection = 1'b0;
           path_cost = 8'd0;
        end
     endcase
end
        */
        assign path_cost      =  (valid_o?(selection?path_cost_1:path_cost_0):8'd0);
        assign valid_o=path_0_valid | path_1_valid;

       /* always_comb begin
           if(valid_o==0) path_cost=8'b0;
                  else if(valid_o==1 && selection==0)path_cost=path_cost_0;
                  else if(valid_o==1 && selection==1)path_cost=path_cost_1;
        end
*/
        always_comb begin
                  if (path_0_valid == 1'b0 && path_1_valid == 1'b1) selection = 1'b1;
```

```
                    else if (path_0_valid == 1'b1 && path_1_valid == 1'b0) selection =
1'b0;
                    else if (path_0_valid == 1'b1 && path_1_valid == 1'b1) begin
                     if (path_cost_0 > path_cost_1) selection = 1'b1;
                     else selection = 1'b0;
                    end
                    else selection=1'b0;
          end


endmodule
```

e. mem_1x1024

```
/*make the data path K bits wide for mem_Kx1024
  K=8 for module mem, K=1 for module mem_disp */
module mem_disp                                          (
  input               clk,
  input               wr,   // write enable
  input       [9:0]   addr,//2^10=1024
  input               d_i,          // data
  output logic          d_o);

  logic               mem   [0:1023];

  always @ (posedge clk) begin
/*
  write synchronously to memory core if enabled
  read synchronously at all times (equiv. to DFF at mem data out)
*/
                  if(wr)begin
                          mem[addr] <= d_i;
                  end
                  d_o <= mem[addr];
  end
endmodule
```

f. mem_8x1024

```
module mem (
  input               clk,
  input               wr,     // write enable
  input       [9:0]   addr,   // 2^10=1024
  input       [7:0]   d_i,    // data input (8-bit)
  output logic  [7:0]   d_o     // data output (8-bit)
);

  logic [7:0] mem [0:1023];      // 1024 x 8-bit memory array

  always @ (posedge clk) begin
    // Write synchronously to memory core if enabled
    // Read synchronously at all times (equiv. to DFF at mem data out)
```

```
      if(wr) begin
        mem[addr] <= d_i;
      end
      d_o <= mem[addr];
    end
  endmodule
```

g. tbu

```
  module tbu (
    input             clk,
    input             rst,
    input             enable,
    input             selection,
    input      [7:0]  d_in_0,
    input      [7:0]  d_in_1,
    output logic      d_o,
    output logic      wr_en
  );

    logic      d_o_reg;
    logic      wr_en_reg;

    logic [2:0] pstate;
    logic [2:0] nstate;

    logic      selection_buf;

    always @(posedge clk) begin
      selection_buf <= selection;
      wr_en        <= wr_en_reg;
      d_o          <= d_o_reg;
    end
  ///reset p state to 0
  //exception:  rst = enable = selection_buf = 1, but selection = 0, then go to n_state
    always @(posedge clk, negedge rst) begin
      if (!rst) begin
        pstate <= 3'd0;
      end else if (enable && selection_buf && !selection) begin
        pstate <= nstate;
                end else if (enable) begin
      pstate <= nstate;
      end
    end

    always_comb begin
        wr_en_reg = selection;
        //d_o_reg = selection ? d_in_1[pstate] : d_in_0[pstate];
        if(selection)d_o_reg=d_in_1[pstate];
                                //else if(!selection) d_o_reg=d_in_0[pstate];
```

```
                else d_o_reg=0;

    case (pstate)
        3'd0: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd0; else nstate = 3'd1;
            end else begin
                if (d_in_0[pstate] == 1'b0) nstate = 3'd0; else nstate = 3'd1;
            end
        end
        3'd1: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd3; else nstate = 3'd2;
            end else begin
                if (d_in_0[pstate] == 1'b0) nstate = 3'd3; else nstate = 3'd2;
            end
        end
        3'd2: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd4; else nstate = 3'd5;
            end else begin
                if (d_in_0[pstate] == 1'b0) nstate = 3'd4; else nstate = 3'd5;
            end
        end
        3'd3: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd7; else nstate = 3'd6;
            end else begin
                if (d_in_0[pstate] == 1'b0) nstate = 3'd7; else nstate = 3'd6;
            end
        end
        3'd4: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd1; else nstate = 3'd0;
            end else begin
                if (d_in_0[pstate] == 1'b0) nstate = 3'd1; else nstate = 3'd0;
            end
        end
        3'd5: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd2; else nstate = 3'd3;
            end else begin
                if (d_in_0[pstate] == 1'b0) nstate = 3'd2; else nstate = 3'd3;
            end
        end
        3'd6: begin
            if (selection_buf) begin
                if (d_in_1[pstate] == 1'b0) nstate = 3'd5; else nstate = 3'd4;
```

```
                end else begin
                    if (d_in_0[pstate] == 1'b0) nstate = 3'd5; else nstate = 3'd4;
                end
            end
            3'd7: begin
                if (selection_buf) begin
                    if (d_in_1[pstate] == 1'b0) nstate = 3'd6; else nstate = 3'd7;
                end else begin
                    if (d_in_0[pstate] == 1'b0) nstate = 3'd6; else nstate = 3'd7;
                end
            end
            default: begin
                nstate = 3'd0;
            end
        endcase
    end

endmodule
```